Lab 1:

In Java, a class can extend only one abstract class but can implement many interfaces, so multiple inheritance of type is achieved with interfaces, not abstract classes.

Prefer an abstract class when:

1. we want to share common state and some default method implementations.
2. Classes have a clear "is-a" relationship and are closely related.

Prefer an interface when:

1. we want to define a contract without shared state.
2. We want a class to play multiple roles, eg., Comparable, Serializable, Runnable, so it can implement several interfaces.

Lab 2:

Encapsulation hides data inside a class using private fields and allows only through controlled public methods, which validate input and prevent illegal changes.

Example:

```
class BankAccount {
    private String accountNumber;
    private double balance;

Public void setAccountNumber(String accNo){
    if(accNo == null || accNo.trim().isEmpty())
        throw new IllegalArgumentException("
            Invalid Account Number");
    this.account Number = accNo;
}
```

```
Public void setInitialBalance(double amount){
    if (amount < 0)
        throw new IllegalArgumentException("
            Balance cannot be negative");
    this.balance = amount;
}

public double getBalance(){
        return balance;
    }
}
```

Here accNo and amount cannot be changed directly from outside, so all updates go through validation logic, preserving data integrity.

Lab 3:

Classes and Responsibilities :

⇒ RegistarParking → Parking request

⇒ ParkingPool → Shared synchronized queue.

⇒ ParkingAgent → worker thread.

⇒ MainClass → Simulates Car arrival.

Code:

```
import java.util.*;

Class RegistarParking {
    String CarNo;
    RegistarParking (String CarNO) {
        this.CarNO = CarNO;
        System.out.println("Car" + CarNo + "requested
            Parking.");
    }
}
```

```
Class ParkingPool {
  Queue<RegisterParking> queue = new LinkedList<>();
  Synchronized void addCar(RegisterParking car){
    queue.add(car);
    notify();
  }
  Synchronized RegisterParking parkCar()
  throws InterruptedException{
    while(queue.isEmpty()) wait();
    return queue.poll();
  }
}

class ParkingAgent extends Thread {
    ParkingPool pool, String name){
        this.pool = pool;
        this.agentName = name;
    }
  public void run() {
  try {
    RegisterParking car = pool.parkCar();
      System.out.println(agentname + "Parked car" +
                car.carNo + "." );
```

```
} catch (Exception e) { }
    }
}

Public class MainClass {
    Public static void main(String[] args) {
    ParkingPool pool = new ParkingPool();
        new ParkingAgent(Pool, "Agent 1").start();
        new ParkingAgent(Pool, "Agent 2").start();
        pool.addCar(new RegisterParking)("PBC123");
        pool.addCar(new RegisterParking)("XYZ456");
    }
}
```

Lab 4:

How JDBC Works:

Java App → JDBC API → Driver → Database → Result.

steps:

1. Load driver
2. Get connection
3. Create Statement
4. Execute query
5. Process ResultSet
6. Close resources.

Code:

```
Connection con = null;
Statement st = null;
ResultSet rs = null;

try {
  Con = DriverManager.getConnection("jdbc:
    mysql://localhost/db", "root", " ");
  St = Con.createStatement();
  rs = St.execteQuery("Select * from Student");
```

```
while (rs.next()) {
    System.out.println(rs.getInt("id") + " " +
    rs.getString("name"));
  }
} catch (SQLException e) {
    e.printStackTrace();
  } finally {
  try { if (rs != null) rs.close(); }
      catch (Exception e) { }

  try { if (st != null) st.close(); }
      catch (Exception e) { }

  try { if (con != null) con.close(); }
      catch (Exception e) { }
  }
```

Lab 5:

Controller Role:

1. Receives request
2. Calles model
3. Sends data to view(JSP)

Servlet:

```
@WebServlet("/student")
Public Class StudentServlet extends HttpServlet{
  Protected void doGet(HttpServletRequest
  req, HttpServletResponse res)
    throws ServletException, IOException{

    req.setAttribute("name", "Mahmudul");
    RequestDispatcher rd = req.getRequestDispatcher
     ("student.JSP");
     rd.forward(req, res);

  }}
```

JSP:

```
<h2> Welcome ${name} </h2>
```

Lab 6 :

Prepared Statement precompiles the sql once and lets execute it multiple times with different parameters, which improves performance for repeated queries and avoids SQL injection by separating SQL from data values.

Code:

```
String sql = "Insert into student (name, age)
"values ( ?,? )";
PreparedStatement ps = con.prepareStatement(sql);
ps.setString (1," Rahim');
ps.setInt (2,20);
ps.executeUpdate();
```

Lab 7:

ResultSet is a cursor-like Object that holds the rows returned by a SELECT statement; you move row by row using next() and read column values with getters like getString() and getInt().

next(): moves the cursor to the ~~next~~ row, returns false when there is no more row.

getString(ColumnName or index): reads a text value from the current row.

getInt(ColumnName or index): reads an integer value from the current row.

Code:

```
String sql = "select id, name, age FROM
                students";
try (Connection conn = DriverManager.getConnection(
    "jdbc:mysql://localhost:3306/testdb", "root",
    "password");
```

```
Statement stmt = conn.createStatement();
ResultSet rs = stmt.executeQuery(sql);{

while (rs.next()) {
    int id = rs.getInt("id");
    String name = rs.getString("name");
    int age = rs.getInt("age");
    System.out.println(id + " - " + name " - " +
        age);
}
Catch (SQLException e) {
    e.printStackTrace();
}
```